LAB ASSIGNMENT 6

LAB ASSIGNMENT 5
TEAM MEMBER DETAILS
MEMBER_1 :KARAN (LEADER)
COLLEGE_ID :20261651048
MEMBER_2 :HIMANSHU BAJPAI
COLLEGE_ID :20261651041
MEMBER_3 :DIVYANSHU TIWARI
COLLEGE_ID :20261651034
MEMBER_4 : ETNI HARSHITHA
COLLEGE_ID :20261651036

LAB ASSIGNMENT 6 — NOTES ANSWERS

Campus Event Board — IIIT Vadodara

1. let vs const — when do you use each, and why avoid var?

In JavaScript, const is used when a variable should not be reassigned after declaration. For example, DOM elements and fixed values are declared using const.

let is used when the value of a variable needs to change during program execution. For example, the events array is declared using let because its value is updated after fetching data from the JSON file.

We avoid var because it is function-scoped rather than block-scoped and allows redeclaration. This can cause unexpected errors and make programs difficult to maintain.

Example:

```
const pageTitle = "Campus Event Board";

let events = [];
```

2. List the data types you used and give one value of each.

The following JavaScript data types and structures are used in this project:

- **String:** "Campus Event Board"
- **Number:** 30
- **Boolean:** false

- **Object:** An individual event record containing title, date, seats and other details.
- **Array:** events, which stores multiple event objects.
- **Set:** rsvpedEventIds, which stores the IDs of events for which the user has RSVP'd.
- **Function:** formatEvent, which formats event information.

Example:

```
const pageTitle = "Campus Event Board";

const exampleSeats = 30;

const exampleIsOnline = false;

const rsvpedEventIds = new Set();
```

3. Show one event object from your array. Why is an array of objects a good model for this page?

Example of an event object from events.json:

```
{
  "id": "ai-ml-workshop",
  "title": "AI & Machine Learning Workshop",
  "date": "2026-11-20",
  "seats": 30,
  "online": false
}
```

An array of objects is a good model because each event can store related information in one object. Multiple event objects can be stored in an array and processed using loops and array methods.

It also makes the website easier to maintain because event cards are generated dynamically from JSON data instead of writing each card manually in HTML.

4. Paste your control-flow rule from A4. What labels did it produce for your events?

The project uses an if / else if / else statement to check the remaining seats and assign an event status.

```
function getEventStatus(seats) {
  if (seats === 0) {
```

```
    return "Full";
  } else if (seats < 5) {
    return "Few left";
  } else {
    return "Open";
  }
}
```

The initial sample event data produces the following results:

1. AI & Machine Learning Workshop — Open (30 seats).
2. CodeSprint Challenge — Full (0 seats).
3. Campus Cultural Evening — Few left (3 seats).

The RSVP feature can change seat counts in memory, so the displayed status may change after an RSVP.

5. Show one function (A6) and one arrow function with an array method (A7). What does each return?

A6 — Function with a parameter and return value

```
function formatEvent(ev) {
  const mode = ev.online ? "Online" : "In person";
  return `${ev.title} — ${formatDate(ev.date)} — ${mode}`;
}
```

This function accepts an event object as a parameter and returns a readable one-line string containing the event title, date and attendance mode.

A7 — Arrow function with an array method

```
const getAvailableEvents = (eventArray) =>
  eventArray.filter((ev) => ev.seats > 0);
```

This arrow function uses the filter() array method. It returns a new array containing only events that have more than zero available seats.

6. Which elements did you select, and with which methods? Why prefer querySelector?

The project uses the following DOM selection methods:

querySelector():

const eventGrid = document.querySelector("#eventGrid");

const searchInput = document.querySelector("#eventSearch");

const menuButton = document.querySelector("#menuButton");

It selects the first element matching a CSS selector.

querySelectorAll():

const allEventSections = document.querySelectorAll(".events");

It selects all elements matching the specified CSS selector.

getElementById():

const eventForm = document.getElementById("eventForm");

It selects an element using its unique ID.

I prefer querySelector() because it supports CSS selector syntax. We can select elements using IDs, classes, attributes and nested selectors. It makes DOM selection flexible and readable.

7. Explain event bubbling in one or two lines, using event.target from B5.

Event bubbling is a process in which an event triggered on a child element propagates upward through its parent elements.

In this project, a click listener is attached to the event grid. The event.target property identifies the actual clicked element, and closest("button") helps identify the button. This allows one event listener to handle clicks on multiple dynamically generated event cards. This technique is called event delegation.

Example:

```
eventGrid.addEventListener("click", (event) => {
  const button = event.target.closest("button");

  if (!button) return;

  console.log("Clicked button:", button);
});
```

8. Describe the three states your fetch handles, and how the user sees each.

The project uses fetch() with async/await and try/catch to load event information from events.json.

It handles three states:

1. Loading State

When the website starts fetching data, it displays a loading message:

Loading…

2. Success State

When the JSON file loads successfully, the response is converted into JavaScript data and event cards are dynamically displayed on the page.

The website also displays the number of events loaded.

3. Error State

If the JSON file cannot be loaded or an error occurs, the catch block handles the error and displays a friendly message:

Events could not be loaded. Run this page using Live Server or a local web server and check that events.json exists.

The error details are also logged in the browser Console.

9. Show your built-in validation attributes and your custom setCustomValidity rule. Why must the server validate again?

The event submission form uses built-in HTML validation attributes:

- required — makes a field compulsory.
- type="email" — checks that the email has a valid format.
- minlength="2" — requires a minimum number of characters in the organizer name.
- pattern="[0-9]{10}" — accepts exactly 10 digits for the contact number.

The project also uses JavaScript's setCustomValidity() method to implement custom validation rules.

Example:

```
organizerName.setCustomValidity(
  organizerName.value.trim().length >= 2
    ? ""
    : "Enter an organizer name with at least two non-space characters."
```

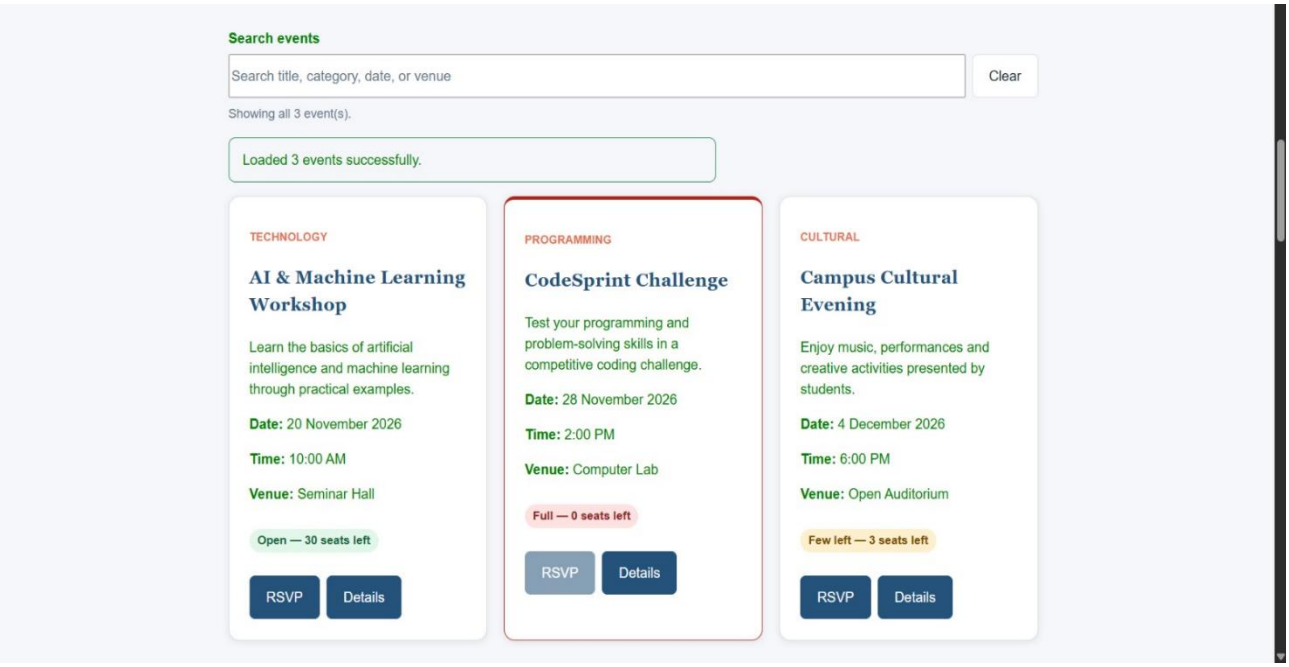
);

The project uses checkValidity() to check the form, reportValidity() to display validation messages and preventDefault() to stop invalid form submission.

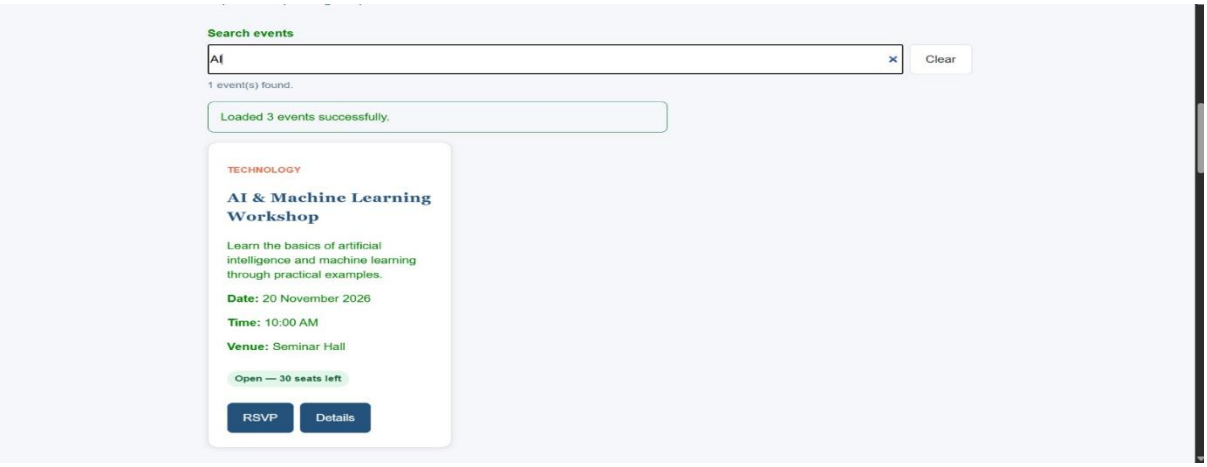
Server-side validation is also necessary because client-side validation can be bypassed or disabled by users. The server must independently validate submitted data to protect data integrity and security.

2. Screenshots – Lab Assignment 6

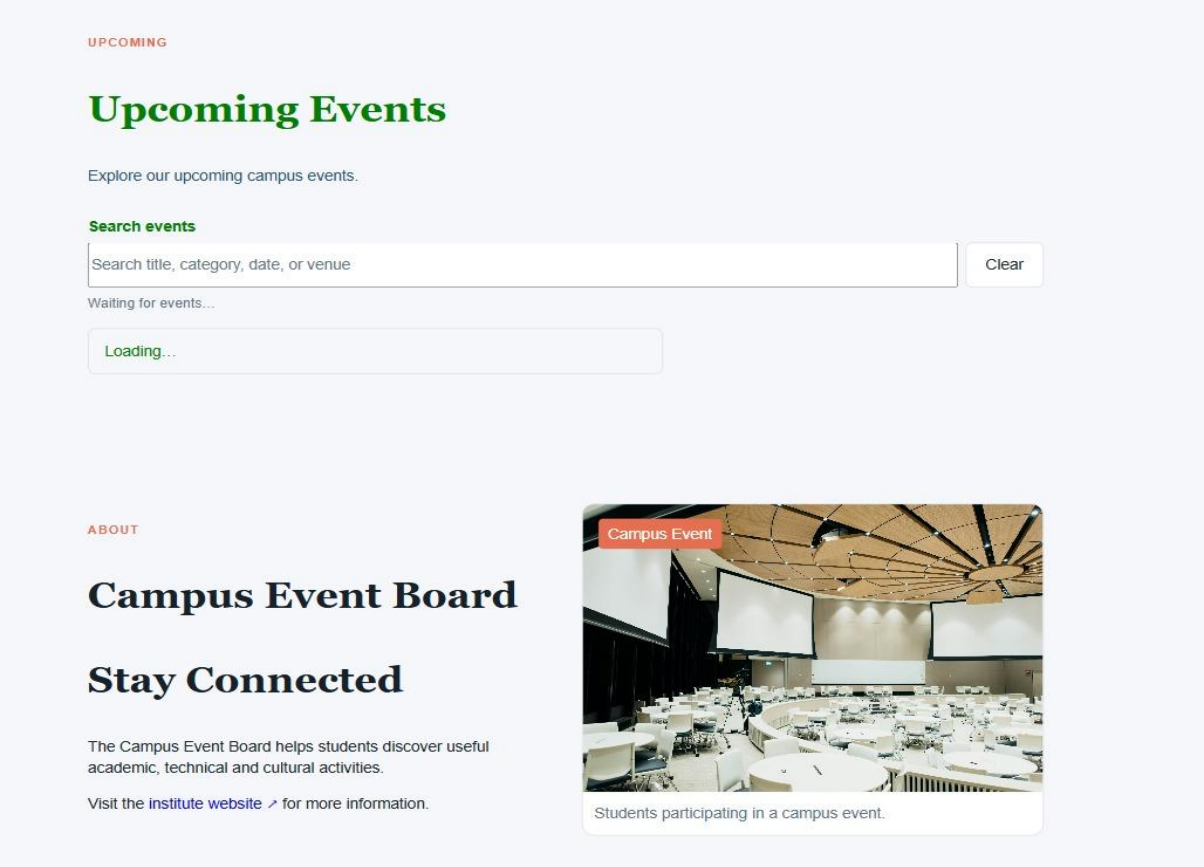
Screenshot 1: Events Rendered from JSON



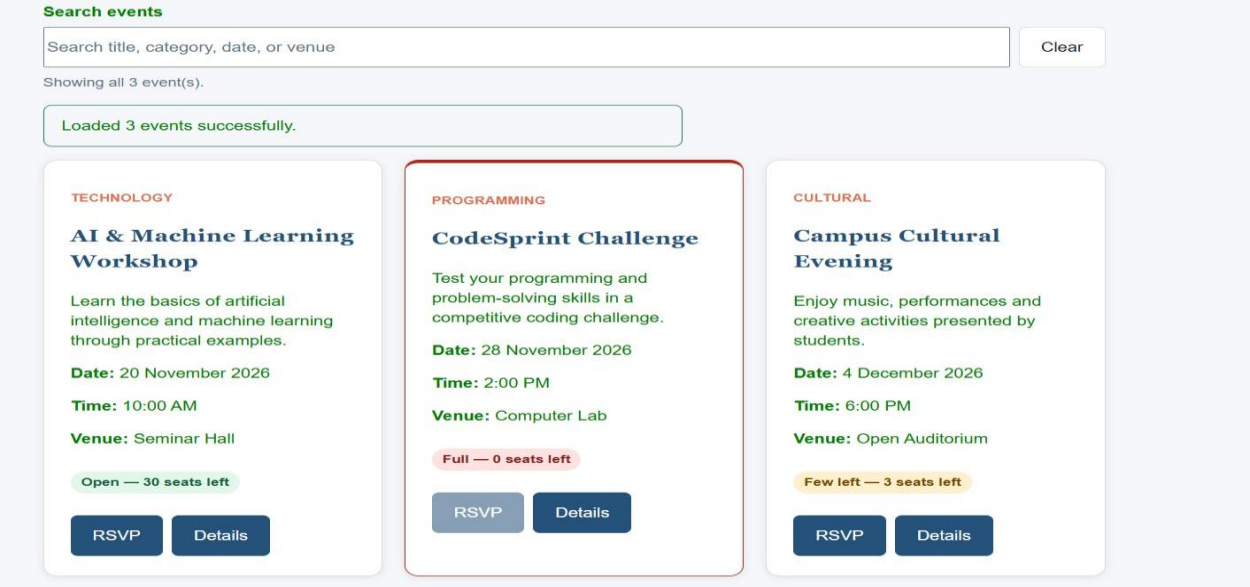
Screenshot 2: Search Filter Working



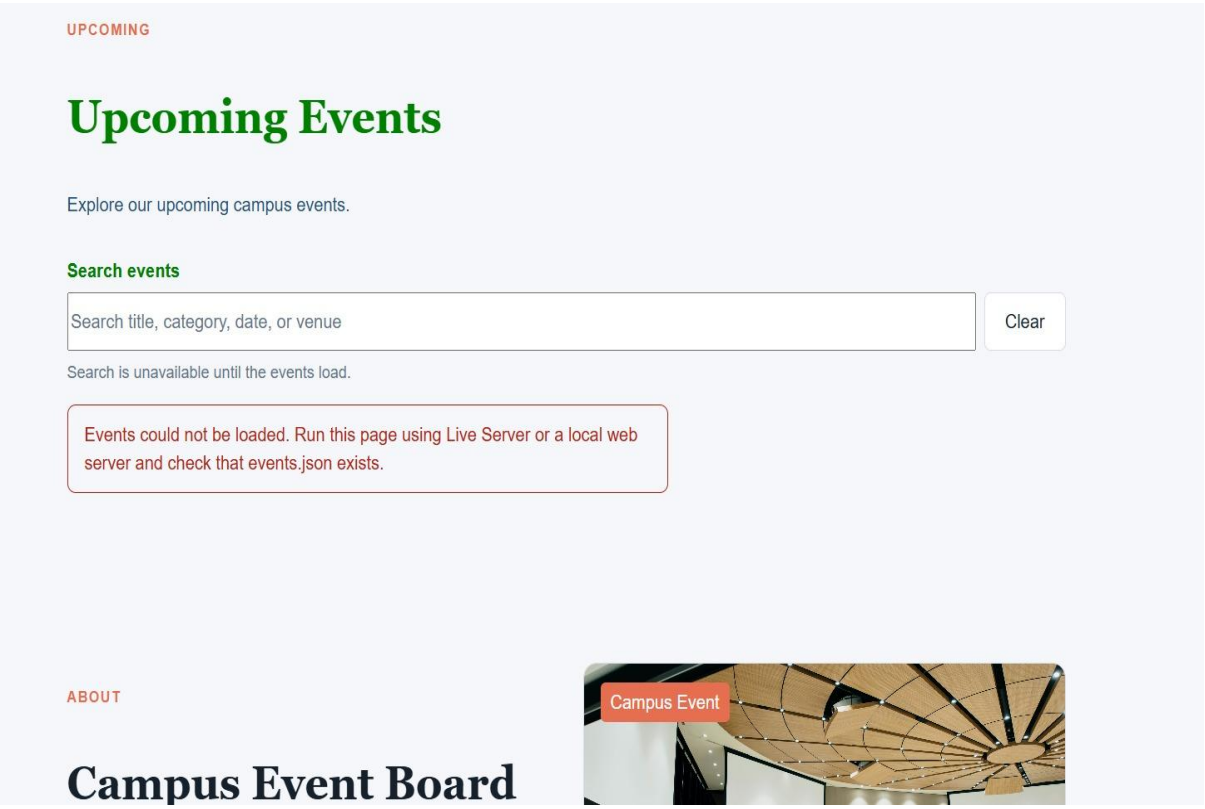
Screenshot 3: Fetch Loading State



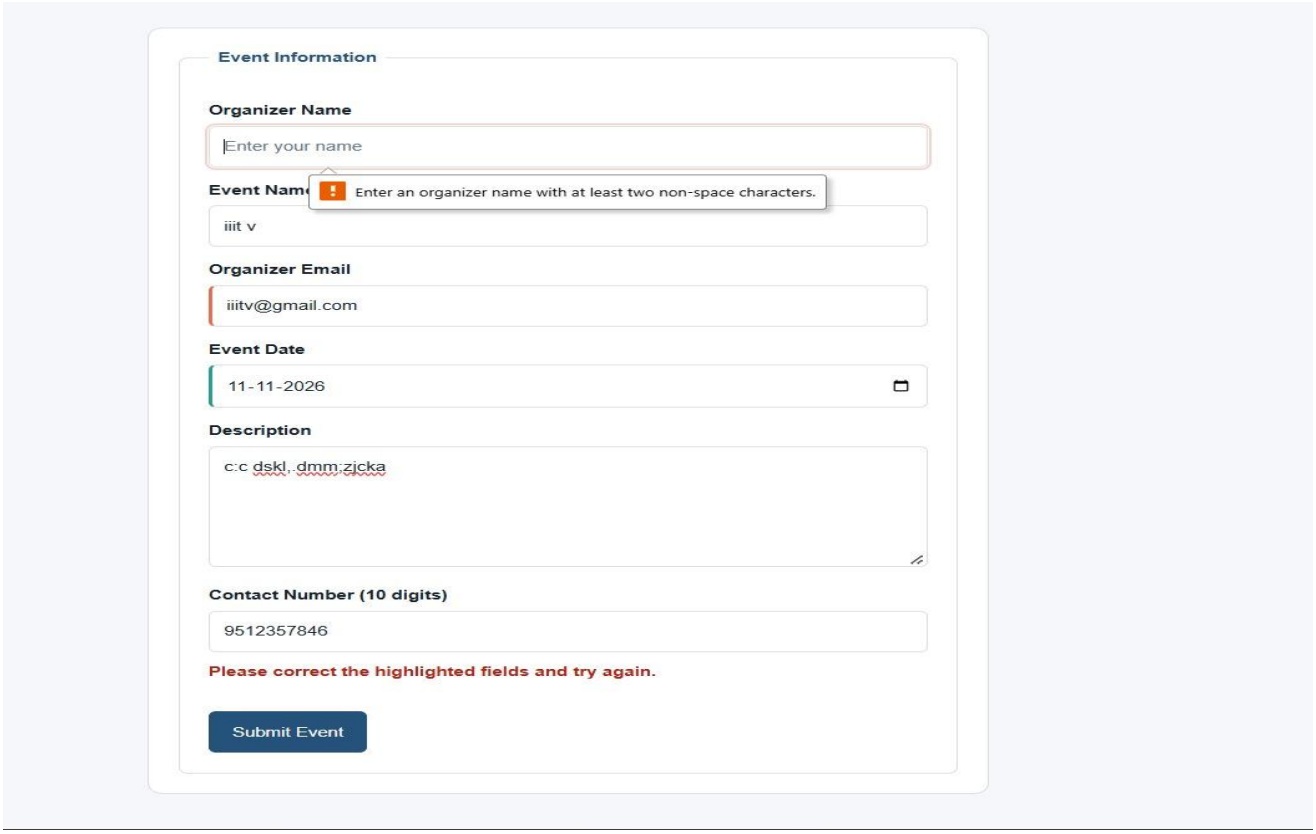
Screenshot 4: Fetch Success State



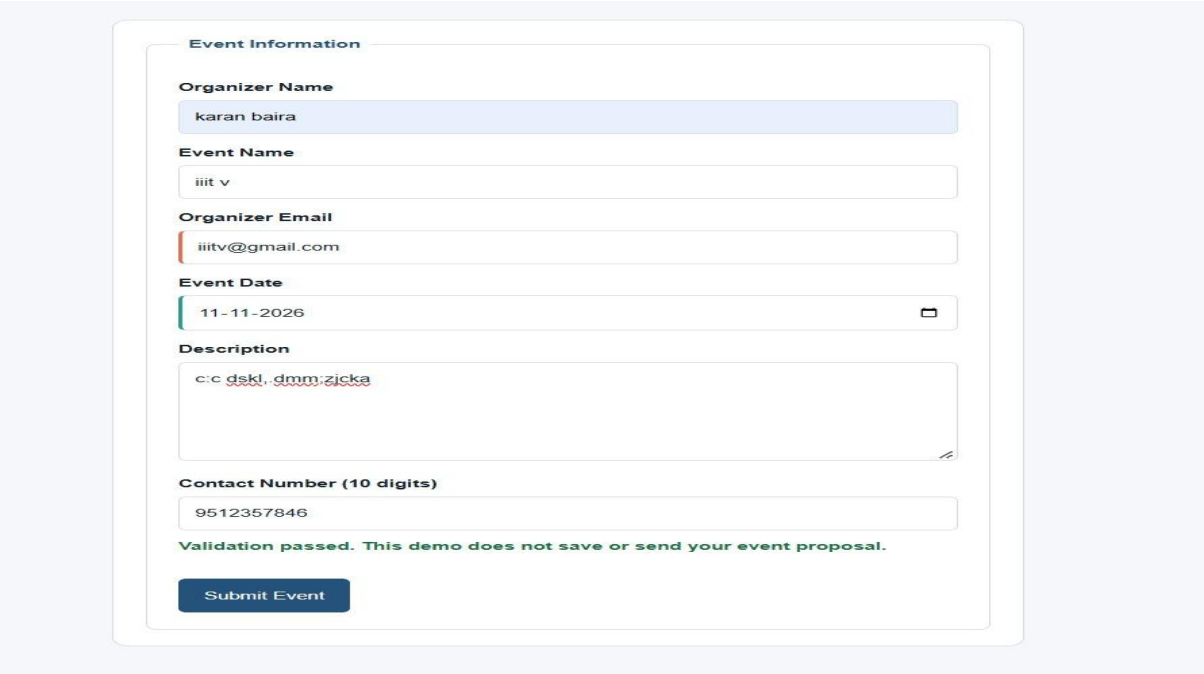
Screenshot 5: Fetch Error State



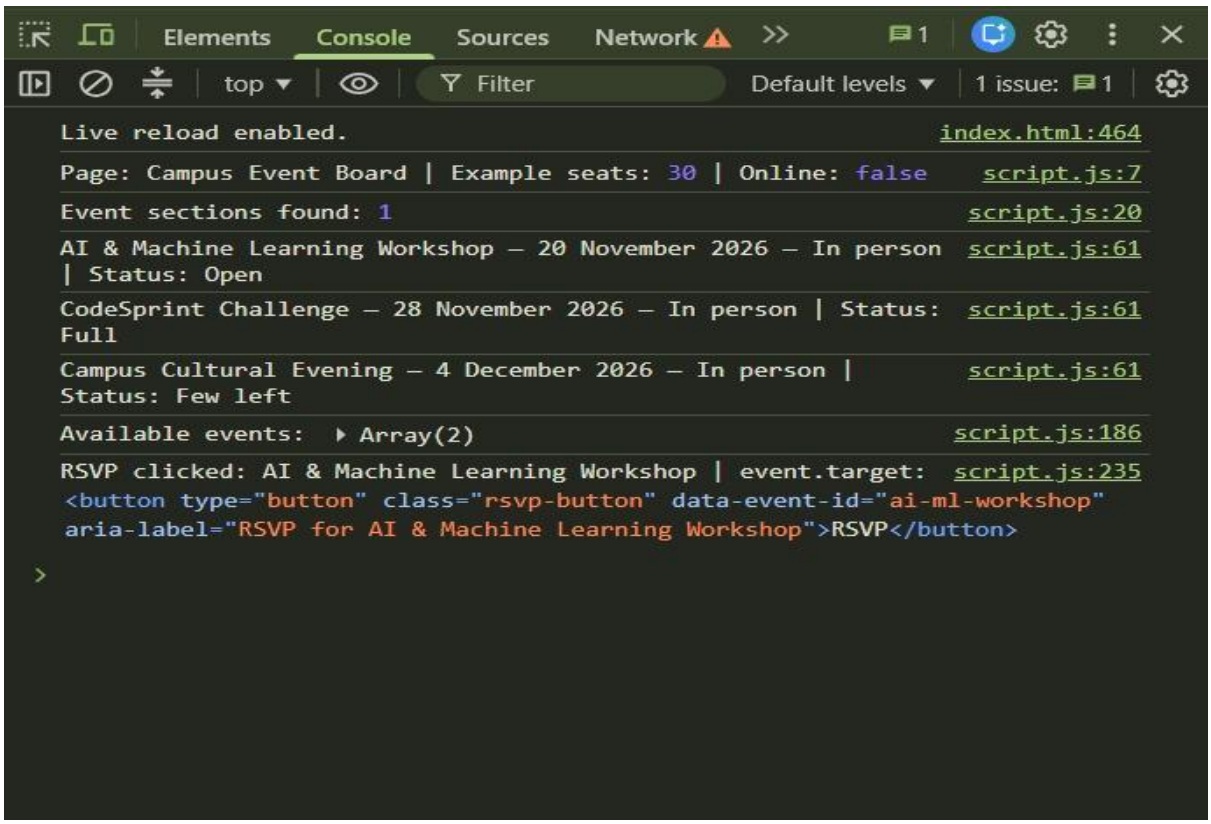
Screenshot 6: Form Validation Error



Screenshot 7: Form Validation Passed



Screenshot 8: JavaScript Console Output



GitHub Repository:

https://github.com/karanbaira5/event-board

GitHub Pages:
https://karanbaira5.github.io/event-board/

README: The project contains a README.md file explaining the project structure, features and instructions for running the website.

Commit History: The GitHub repository contains multiple commits with descriptive commit messages, including the Lab Assignment 6 changes.

JavaScript Code:

```
/* LAB ASSIGNMENT 6 — JavaScript for the Campus Event Board */

/* A2: use const by default; use let only for values that will be reassigned.
*/
const pageTitle = "Campus Event Board"; // string
const exampleSeats = 30;                // number
const exampleIsOnline = false;          // boolean
console.log("Page:", pageTitle, "| Example seats:", exampleSeats, "| Online:",
exampleIsOnline);

/* B1: select elements using querySelector, querySelectorAll, and
getElementById. */
const eventGrid = document.querySelector("#eventGrid");
const eventStatus = document.querySelector("#eventStatus");
const searchInput = document.querySelector("#eventSearch");
const searchStatus = document.querySelector("#searchStatus");
const clearSearchButton = document.querySelector("#clearSearch");
const menuButton = document.querySelector("#menuButton");
const navLinks = document.querySelector("#navLinks");
const eventForm = document.getElementById("eventForm");
const formMessage = document.querySelector("#formMessage");
const allEventSections = document.querySelectorAll(".events");
console.log("Event sections found:", allEventSections.length);

/* A3: events is loaded from events.json; it is the source of truth for cards.
*/
let events = [];
```

```
let visibleEvents = [];
const rsvpedEventIds = new Set();

/* A4: control flow labels an event according to its remaining seats. */
function getEventStatus(seats) {
  if (seats === 0) {
    return "Full";
  } else if (seats < 5) {
    return "Few left";
  } else {
    return "Open";
  }
}

/* A6: function with a parameter and a return value. */
function formatEvent(ev) {
  const mode = ev.online ? "Online" : "In person";
  return `${ev.title} — ${formatDate(ev.date)} — ${mode}`;
}

/* Helper function: convert the JSON date to a readable date. */
function formatDate(dateString) {
  const date = new Date(`${dateString}T12:00:00`);
  return new Intl.DateTimeFormat("en-IN", {
    day: "numeric",
    month: "long",
    year: "numeric"
  }).format(date);
}

/* A7: arrow function and array method filter() create a new available-events
array. */
const getAvailableEvents = (eventArray) =>
  eventArray.filter((ev) => ev.seats > 0);

/* A5: for...of processes every event once. Local ev is scoped to this loop.
*/
function logEvents(eventArray) {
  for (const ev of eventArray) {
    console.log(formatEvent(ev), "| Status:", getEventStatus(ev.seats));
  }
}

/* B2/B3: build each card with createElement, textContent, classList and
attributes. */
function createEventCard(ev) {
  const article = document.createElement("article");
  article.classList.add("event-card");
```

```
  article.dataset.eventId = ev.id;

  if (ev.seats === 0) {
    article.classList.add("full");
  }

  const category = document.createElement("p");
  category.classList.add("event-type");
  category.textContent = ev.category;

  const title = document.createElement("h3");
  title.textContent = ev.title;

  const description = document.createElement("p");
  description.textContent = ev.description;

  const date = document.createElement("p");
  date.classList.add("event-date");
  const dateLabel = document.createElement("strong");
  dateLabel.textContent = "Date: ";
  const time = document.createElement("time");
  time.dateTime = ev.date;
  time.textContent = formatDate(ev.date);
  date.append(dateLabel, time);

  const eventTime = document.createElement("p");
  const timeLabel = document.createElement("strong");
  timeLabel.textContent = "Time: ";
  eventTime.append(timeLabel, document.createTextNode(ev.time));

  const venue = document.createElement("p");
  const venueLabel = document.createElement("strong");
  venueLabel.textContent = "Venue: ";
  venue.append(venueLabel, document.createTextNode(ev.venue));

  const status = document.createElement("p");
  status.classList.add("seat-status");
  const label = getEventStatus(ev.seats);
  status.textContent = `${label} — ${ev.seats} seat${ev.seats === 1 ? "" :
"s"} left`;
  status.setAttribute("aria-label", `Registration status: ${label}`);
  if (label === "Full") status.classList.add("full");
  if (label === "Few left") status.classList.add("few-left");

  const actions = document.createElement("div");
  actions.classList.add("event-actions");

  const rsvp = document.createElement("button");
```

```
  rsvp.type = "button";
  rsvp.classList.add("rsvp-button");
  rsvp.dataset.eventId = ev.id; // B5: data attribute connects the button to
its event.
  rsvp.textContent = rsvpedEventIds.has(ev.id) ? "RSVP added" : "RSVP";
  rsvp.disabled = ev.seats === 0 || rsvpedEventIds.has(ev.id);
  rsvp.setAttribute("aria-label", `RSVP for ${ev.title}`);

  const details = document.createElement("button");
  details.type = "button";
  details.classList.add("details-button");
  details.dataset.eventId = ev.id;
  details.textContent = "Details";
  details.setAttribute("aria-label", `Show details for ${ev.title}`);

  actions.append(rsvp, details);
  article.append(category, title, description, date, eventTime, venue, status,
actions);
  return article;
}

/* B2: render cards from the array; no event card markup is hard-coded in
HTML. */
function renderEvents(eventArray) {
  eventGrid.replaceChildren();

  if (eventArray.length === 0) {
    const empty = document.createElement("p");
    empty.classList.add("empty-state");
    empty.textContent = "No events match your search. Try another keyword.";
    eventGrid.appendChild(empty);
    return;
  }

  for (const ev of eventArray) {
    eventGrid.appendChild(createEventCard(ev));
  }
}

/* B6: filter event data live while the user types. */
function applySearch() {
  const query = searchInput.value.trim().toLowerCase();
  visibleEvents = events.filter((ev) => {
    const searchable = `${ev.title} ${ev.category} ${ev.date}
${formatDate(ev.date)} ${ev.venue} ${ev.description} ${ev.online ? "online" :
"in person"}`.toLowerCase();
    return searchable.includes(query);
  });
```

```
  renderEvents(visibleEvents);
  searchStatus.textContent = query
    ? `${visibleEvents.length} event(s) found.`
    : `Showing all ${visibleEvents.length} event(s).`;
}

/* B7: fetch + await, with Loading, success and error states. */
async function loadEvents() {
  eventStatus.className = "event-status";
  eventStatus.textContent = "Loading…";
  eventGrid.replaceChildren();

  try {
    const response = await fetch("events.json");
    if (!response.ok) {
      throw new Error(`Could not load events.json (HTTP ${response.status})`);
    }

    const data = await response.json();
    if (!Array.isArray(data)) {
      throw new Error("events.json must contain an array of events.");
    }

    events = data;
    visibleEvents = [...events];
    logEvents(events);
    console.log("Available events:", getAvailableEvents(events));

    eventStatus.className = "event-status success";
    eventStatus.textContent = `Loaded ${events.length} events successfully.`;
    applySearch();
  } catch (error) {
    console.error("Unable to load events:", error);
    eventStatus.className = "event-status error";
    eventStatus.textContent = "Events could not be loaded. Run this page using
Live Server or a local web server and check that events.json exists.";
    searchStatus.textContent = "Search is unavailable until the events load.";
    eventGrid.replaceChildren();
  }
}

/* B4: hamburger button toggles the navigation links. */
menuButton.addEventListener("click", () => {
  const isOpen = navLinks.classList.toggle("is-open");
  menuButton.setAttribute("aria-expanded", String(isOpen));
  menuButton.textContent = isOpen ? "✕ Close menu" : "☰ Menu";
});
```

```
/* B6: input reacts to typing; Escape clears the search. */
searchInput.addEventListener("input", applySearch);
searchInput.addEventListener("keydown", (event) => {
  if (event.key === "Escape") {
    searchInput.value = "";
    applySearch();
    searchInput.blur();
  }
});
clearSearchButton.addEventListener("click", () => {
  searchInput.value = "";
  applySearch();
  searchInput.focus();
});

/* B5: event.target + bubbling + delegation; one listener handles all card
buttons. */
eventGrid.addEventListener("click", (event) => {
  const button = event.target.closest("button");
  if (!button) return;

  const eventId = button.dataset.eventId;
  const selectedEvent = events.find((ev) => ev.id === eventId);
  if (!selectedEvent) return;

  if (button.classList.contains("rsvp-button")) {
    if (selectedEvent.seats > 0 && !rsvpedEventIds.has(selectedEvent.id)) {
      rsvpedEventIds.add(selectedEvent.id);
      selectedEvent.seats -= 1;
      console.log("RSVP clicked:", selectedEvent.title, "| event.target:",
event.target);
      applySearch();
      eventStatus.className = "event-status success";
      eventStatus.textContent = `Demo RSVP recorded for
${selectedEvent.title}. Seat count updated on this page only.`;
    }
  } else if (button.classList.contains("details-button")) {
    eventStatus.className = "event-status";
    eventStatus.textContent = `${formatEvent(selectedEvent)} —
${selectedEvent.venue}. ${selectedEvent.description}`;
    console.log("Details clicked:", selectedEvent.title, "| event.target:",
event.target);
  }
});

/* B8: built-in validation attributes plus a custom setCustomValidity rule. */
const organizerName = document.querySelector("#organizer-name");
```

```
const contactNumber = document.querySelector("#contact-number");

function validateCustomFields() {
  organizerName.setCustomValidity(
    organizerName.value.trim().length < 2
      ? "Enter an organizer name with at least two non-space characters."
      : ""
  );
  contactNumber.setCustomValidity(
    contactNumber.value && !/^[0-9]{10}$/.test(contactNumber.value)
      ? "Enter exactly 10 digits, without spaces or country code."
      : ""
  );
}

organizerName.addEventListener("input", validateCustomFields);
contactNumber.addEventListener("input", validateCustomFields);

eventForm.addEventListener("submit", (event) => {
  event.preventDefault(); // B8: this front-end demo does not submit to a
server.
  validateCustomFields();
  formMessage.className = "form-message";

  if (!eventForm.checkValidity()) {
    eventForm.reportValidity();
    formMessage.classList.add("error");
    formMessage.textContent = "Please correct the highlighted fields and try
again.";
    console.warn("Form validation failed.");
    return;
  }

  formMessage.classList.add("success");
  formMessage.textContent = "Validation passed. This demo does not save or
send your event proposal.";
  console.log("Form validation passed for:", document.querySelector("#event-
name").value);
});

/* B7: start after the HTML is parsed (script tag uses defer). */
loadEvents();
```

Project: Campus Event Board — IIIT Vadodara
Assignment: CS445 — Lab Assignment 6